

Specification of Ultra-Low Latency Cryptographic Hash

Algorithm LLH

Feng Xiutao, Zeng Xiangyong, Wang Zhangyi, Xu Shengyuan, Xiang Zejun, Wang Huandong, Tang Ming, Tang Deng, Qin Bo, Gao Bo, Zhang Weiguo, Chen Siwei

06-25-2026

Contents

1. Algorithm Overview	3
1.1 Introduction	3
1.2 Notation	3
1.3 Overall Structure	4
1.4 Algorithm Parameters and Instances	4
1.4.1 Algorithm Parameters	4
1.4.2 Recommended Instance Parameters	5
2. Detailed Algorithm Design	5
2.1 Overall Algorithm Structure	5
2.2 Processing Procedure	6
2.2.1 Message Padding	6
2.2.2 Absorbing and Squeezing	6
2.3 Permutation P	7
2.3.1 Internal State Organization	7
2.3.2 Structure of the Permutation P	8
2.4 Round Function RoundFun	9
2.4.1 Column Substitution (SubColumn)	9
2.4.2 Row-wise Cyclic Shift (ShiftRows)	10
2.4.3 Constant Addition (AddConstant)	11
Appendix A. Complete Round Constant Reference Values for Different Values of w and the Corresponding Recommended Numbers of Rounds	12

1. Algorithm Overview

1.1 Introduction

LLH is a family of ultra-low latency cryptographic hash algorithms based on a variant of the sponge construction. It supports input messages whose bit length is at most $2^{64} - 1$ and four output digests of bit lengths 256, 512, 768 and 1024 respectively. For a d -bit output digest, the algorithm can provide $d/2$ -bit collision resistance and d -bit preimage resistance. It is applicable to cryptographic use cases such as digital signatures, message authentication code (MAC) generation, key derivation functions (KDFs), and pseudorandom number generation.

The core of LLH is a permutation function P . This function is designed using bit-slicing techniques and employs ultra-low-latency cryptographic components with good cryptographic properties. Through multiple rounds of iteration on the internal state, the algorithm ensures randomness and unpredictability of the output digest while providing high performance. This specification mainly defines the structure, processing procedure, and parameter requirements of the LLH cryptographic hash algorithm.

1.2 Notation

The notation used in this document is listed in Table 1-1.

Table 1-1. Notation

$\oplus$	Bitwise exclusive-or (XOR).
$\wedge$	Bitwise AND.
$\lll$	Rotation left within a w -bit word.
$\ggg$	Rotation right within a w -bit word.
$\parallel$	Concatenation of bit strings.
0^k	The all-zero bit string of length k .
$S_{i:j}$	The word in state S from bit i to bit j .
σ_i	The rotation offset of the i -th word.
W_i	The i -th word of the internal state, where $i = 0, 1, \dots, 23$.
RC_t	The round constant of round t , with width w bits.

1.3 Overall Structure

LLH adopts a variant of the sponge construction. The construction maintains a b -bit internal state S . The state is divided into two parts:

- **Rate:** the first r bits of S part used to interact with message blocks;
- **Capacity:** the last c bits of S used to ensure algorithmic security and not directly involved in input interaction;

which satisfy $b = r + c$.

The algorithm consists of two phases:

(1) **Absorbing phase:** the padded input message is divided into blocks, which are XORed sequentially with the first r bits of the state. The permutation function P is applied once for each XOR operation.

(2) **Squeezing phase:** the permutation function P is applied, and the first r bits of the result are extracted as the output digest.

1.4 Algorithm Parameters and Instances

1.4.1 Algorithm Parameters

The security strength of LLH is determined by the word size w and the number of rounds N_r . The internal state S is organized as a $24 \times w$ bit matrix, so the total state width is $b = 24 \times w$ bits. The main parameters are defined in Table 1-2.

Table 1-2. Algorithm parameter definitions

Parameter	Name	Description
w	Word Size	The bit width of each row of the internal state.
N_r	Number of Rounds	The number of iterations of the permutation function P .
d	Output Length	The bit length of the final hash digest.

1.4.2 Recommended Instance Parameters

The algorithm supports multiple word-size configurations to meet different performance and security requirements. The standard values of the word size w are 32, 64, 96, and 128 bits. The corresponding internal state sizes are shown in Table 1-3.

Table 1-3. Recommended algorithm parameter configurations

Instance (Variant)	Word size w	State size b	Typical output length d	Rate	Capacity	Rounds N_r
LLH-256	32	768	256	256	512	16
LLH-512	64	1536	512	512	1024	24
LLH-768	96	2304	768	768	1536	32
LLH-1024	128	3072	1024	1024	2048	40

Note: the number in an instance name, such as 256, 512, 768, or 1024, denotes the recommended length of output digest for that configuration. The concrete security strength depends on the capacity c . In general, the security strength against collision attacks is $\min(d/2, c/2)$ bits. The design rule for the number of rounds is $N_r = 8 + \lceil w/4 \rceil$, which ensures sufficient confusion and diffusion.

2. Detailed Algorithm Design

2.1 Overall Algorithm Structure

The overall structure of LLH is shown in Figure 2-1. After padding, the message m enters the absorbing phase, where one message block is absorbed each time through the permutation function P . The algorithm then enters the squeezing phase and generates the output digest by calling the permutation function P .

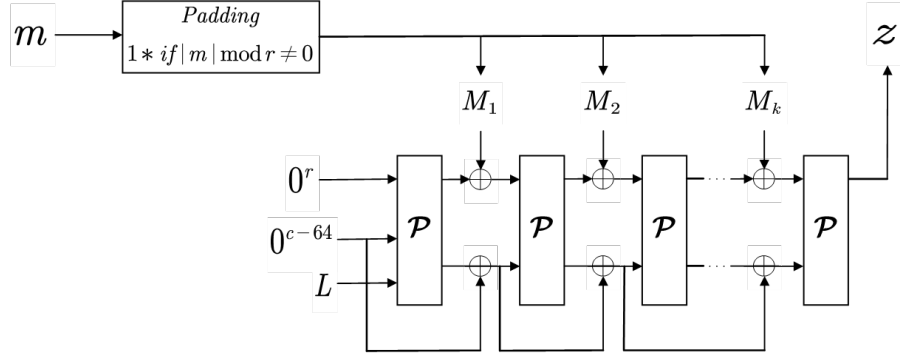


Figure 2-1. Structure of the LLH hash algorithm

2.2 Processing Procedure

2.2.1 Message Padding

Let L be the bit length of the input message M . Before absorbing, M shall be padded so that its length is an integer multiple of the rate r . The padding rule is as follows:

- (1) If L is an integer multiple of r , that is, $L \bmod r = 0$, no padding is applied, and the original message is directly used as the padded message M' .
- (2) If L is not an integer multiple of r , append '1' bits to the end of the message until the length reaches the smallest integer multiple of r . The number of padding bits is $n = r - (L \bmod r)$.

Before being injected into the internal state, the message is encoded as a bit string in the little-endian order. More specifically, within each w -bit word, bytes are placed from the least significant byte position to the most significant byte position in an increasing address order. If the message ends with a partial byte, the valid bits occupy the lower bit positions first, and the remaining higher bit positions are filled with '1' according to the padding rule. For example, when $w = 32$ and the input byte sequence is '0A 01 03 04', the corresponding injected 32-bit word is $0x0403010A$. If the input is the two-bit message '10', interpreted in little-endian bit order as $bit_0 = 1$ and $bit_1 = 0$, then the first byte after padding is $0xFD$. The padded message sequence is denoted by $M' = M_1 \parallel M_2 \parallel \dots \parallel M_k$, where each block M_i has length r bits.

2.2.2 Absorbing and Squeezing

Let the state width be b bits, where $b = r + c$, r is the rate, and c is the capacity.

(1) Initialization:

The initial state is $S^{(0)} = 0^{b-64} \parallel L$, that is, the original message length L is encoded and embedded into the last 64 bits of the initial state, while all other bits are kept as zero.

(2) Absorbing:

For each message block M_i , where $1 \leq i \leq k$, execute sequentially:

$$S^{(i)} = P(S^{(i-1)}) \oplus (M_i \parallel S_{r:b-1}^{(i-1)}).$$

(3) Squeezing:

Compute

$$S^{(k+1)} = P(S^{(k)}),$$

then $Z = S_{0:r-1}^{(k+1)}$ is the final hash digest.

2.3 Permutation P

2.3.1 Internal State Organization

The input of the permutation function P is an internal state S of b bits. Logically, the state S is organized as a $24 \times w$ two-dimensional bit matrix, where w is the word-size parameter:

$$S = \begin{pmatrix} S_{0,0} & S_{0,1} & \cdots & S_{0,w-1} \\ S_{1,0} & S_{1,1} & \cdots & S_{1,w-1} \\ \cdots & \cdots & \cdots & \cdots \\ S_{23,0} & S_{23,1} & \cdots & S_{23,w-1} \end{pmatrix},$$

where $s_{i,j}$ denotes the bit in row i and column j , where $0 \leq i < 24$ and $0 \leq j < w$.

Row: Each row contains w bits, referred to as a word. There are 24 words in total, denoted by $W_0, W_1, \dots, W_{23}$.

Column: Each column contains 24 bits, referred to as a slice. There are w slices in total, denoted by $C_0, C_1, \dots, C_{w-1}$.

State size: Depending on the instance, the total state size b may be 768, 1536, 2304, or 3072 bits, corresponding to $w = 32, 64, 96, 128$, respectively.

During the absorbing phase, an input message block is XORed with the first r bits of the state, namely the first 8 words. During the squeezing phase, the hash digest is generated from the first r bits of the state.

2.3.2 Structure of the Permutation P

The permutation function P is based on bit-slicing techniques. It takes a $24 \times w$ bit state as input and outputs a state of the same length. The permutation function P consists of N_r rounds of the round function **RoundFun**. Each round of **RoundFun** contains three steps: column substitution, row-wise cyclic shift, and constant addition. The process is shown in Figure 2-2:

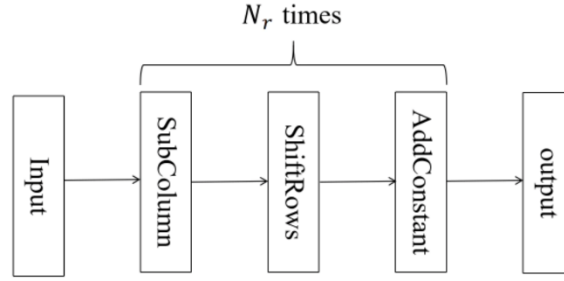


Figure 2-2. Schematic diagram of the permutation P

Let $X^{(t)}$ be the state after the t -th round iteration. The permutation process is expressed as:

$$X^{(0)} = X_{input},$$

$$X^{(t+1)} = RoundFun(X^{(t)}), \quad 0 \leq t < N_r,$$

$$Y_{output} = X^{(N_r)}.$$

Each round function **RoundFun**, where $0 \leq t < N_r$, consists of the following three transformations applied in order:

- (1) Column Substitution (SubColumn): Apply a nonlinear substitution to the internal state column by column.
- (2) Row-wise Cyclic Shift (ShiftRows): Apply cyclic shifts based on w -bit words.
- (3) Constant Addition (AddConstant). XOR a specific constant to remove symmetry.

2.4 Round Function **RoundFun**

2.4.1 Column Substitution (**SubColumn**)

Column substitution is the only nonlinear transformation in the round function **RoundFun**. This operation processes the w slices of the internal state in parallel. Each slice C_j , where $0 \leq j < w$, is transformed by a 24-bit super S-box $\mathcal{S}_{24}$.

2.4.1.1 The Super S-box $\mathcal{S}_{24}$

The super S-box $\mathcal{S}_{24}$ is a transformation from 24 input bits to 24 output bits. Let the 24-bit input vector be $(v_0, v_1, \dots, v_{23})$.

- First, encode the vector as a 6×4 bit matrix $M_{6 \times 4}$ in the row-major order:

$$M_{6 \times 4} = \begin{pmatrix} v_0 & v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 & v_7 \\ v_8 & v_9 & v_{10} & v_{11} \\ v_{12} & v_{13} & v_{14} & v_{15} \\ v_{16} & v_{17} & v_{18} & v_{19} \\ v_{20} & v_{21} & v_{22} & v_{23} \end{pmatrix}.$$

- Second, a 6-bit S-box $\mathcal{S}_6$ is applied to each column of the matrix $M_{6 \times 4}$ (a total of 4 columns, each of 6 bits). It is defined as:

$$\mathcal{S}_6(x) = x \oplus (x \ggg 2) \oplus ((x \ggg 1) \wedge (x \ggg 3)) \oplus 0x1B,$$

where all operations are performed on 6-bit words.

- Third, apply the derangement to the result after the column operation. The shuffle rule is shown in Table 2-3:

Table 2-3. Derangement in $\mathcal{S}_{24}$

0	1	2	3	derangement →	4	9	18	23
4	5	6	7		8	1	22	15
8	9	10	11		12	17	6	3
12	13	14	15		20	5	10	19
16	17	18	19		0	21	14	11
20	21	22	23		16	13	2	7

- Finally, each row of the matrix after the derangement is regarded as a 4-bit row vector R_i ,

where $0 \leq i < 6$, and is right-multiplied by a fixed 4×4 matrix D . Let the transformed row vector be R'_i . Then

$$R'_i = R_i \times D \pmod{2},$$

where the matrix D is defined as:

$$D = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}.$$

2.4.2 Row-wise Cyclic Shift (ShiftRows)

After column substitution, the 24 words W_i of the internal state, equivalently the rows, are cyclically shifted. Let the i -th word W_i , where $0 \leq i < 24$, be cyclically shifted left by σ_i positions, then

$$W_i \leftarrow W_i \lll \sigma_i.$$

For the values of σ_i corresponding to different word sizes w , please refer to Table 2-4.

Table 2-3. Specific Values for Circular Row Shifts

i	$w=32$	$w=64$	$w=96$	$w=128$
0	3	2	2	1
1	16	25	11	16
2	13	20	28	55
3	26	51	53	118
4	8	15	19	10
5	29	40	34	29
6	2	37	57	72
7	23	6	88	11
8	21	36	52	35
9	18	63	73	58
10	31	62	6	105
11	28	33	43	48
12	10	1	5	76
13	15	30	32	103
14	4	31	67	26
15	9	4	14	101
16	7	38	70	5
17	20	5	7	36

18	17	8	48	91
19	30	47	1	42
20	12	19	55	78
21	1	52	94	113
22	6	57	45	44
23	27	34	4	127

2.4.3 Constant Addition (AddConstant)

This operation XORs a round constant RC_t only to the last word W_{23} of the internal state, that is,

$$W_{23} \leftarrow W_{23} \oplus RC_t,$$

where RC_t is the w -bit round constant of round t , where $0 \leq t < N_r$.

The round constant sequence $\{RC_0, RC_1, \dots, RC_{N_r-1}\}$ is determined by the natural mathematical constant e (Euler's number, $e \approx 2.71828 \dots$) as follows:

1. Take the fractional part: Take the fractional part of the natural constant e , namely $e - 2$;
2. Binary conversion: Convert this fractional part into a binary bit stream;
3. Segmented extraction: Starting from bit position 0 of the bit stream, extract w bits for each round in sequence. That is, the first round $t = 0$ uses bits 0 through $w - 1$, the second round $t = 1$ uses bits w through $2w - 1$, and so on.

Appendix A lists reference constants generated by the above rule for different values of w .

Appendix A. Complete Round Constant Reference Values for Different Values of w and the Corresponding Recommended Numbers of Rounds

Table A-1. Round constants for $w = 32$, rounds 0~15

Round t	Round constant RC
0	$RC_0 = 0xB7E15162$
1	$RC_1 = 0x8AED2A6A$
2	$RC_2 = 0xBF715880$
3	$RC_3 = 0x9CF4F3C7$
4	$RC_4 = 0x62E7160F$
5	$RC_5 = 0x38B4DA56$
6	$RC_6 = 0xA784D904$
7	$RC_7 = 0x5190CFEF$
8	$RC_8 = 0x324E7738$
9	$RC_9 = 0x926CFBE5$
10	$RC_{10} = 0xF4BF8D8D$
11	$RC_{11} = 0x8C31D763$
12	$RC_{12} = 0xDA06C80A$
13	$RC_{13} = 0xBB1185EB$
14	$RC_{14} = 0x4F7C7B57$
15	$RC_{15} = 0x57F59584$

Table A-2. Round constants for $w = 64$, rounds 0~23

Round t	Round constant RC
0	$RC_0 = 0xB7E151628AED2A6A$
1	$RC_1 = 0xBF7158809CF4F3C7$
2	$RC_2 = 0x62E7160F38B4DA56$
3	$RC_3 = 0xA784D9045190CFEF$
4	$RC_4 = 0x324E7738926CFBE5$

5	$RC_5 = 0xF4BF8D8D8C31D763$
6	$RC_6 = 0xDA06C80ABB1185EB$
7	$RC_7 = 0x4F7C7B5757F59584$
8	$RC_8 = 0x90CFD47D7C19BB42$
9	$RC_9 = 0x158D9554F7B46BCE$
10	$RC_{10} = 0xD55C4D79FD5F24D6$
11	$RC_{11} = 0x613C31C3839A2DDF$
12	$RC_{12} = 0x8A9A276BCFBFA1C8$
13	$RC_{13} = 0x77C56284DAB79CD4$
14	$RC_{14} = 0xC2B3293D20E9E5EA$
15	$RC_{15} = 0xF02AC60ACC93ED87$
16	$RC_{16} = 0x4422A52ECB238FEE$
17	$RC_{17} = 0xE5AB6ADD835FD1A0$
18	$RC_{18} = 0x753D0A8F78E537D2$
19	$RC_{19} = 0xB95BB79D8DCAEC64$
20	$RC_{20} = 0x2C1E9F23B829B5C2$
21	$RC_{21} = 0x780BF38737DF8BB3$
22	$RC_{22} = 0x00D01334A0D0BD86$
23	$RC_{23} = 0x45CBFA73A6160FFE$

Table A-3. Round constants for $w = 96$, rounds 0~31

Round t	Round constant RC
0	$RC_0 = 0xB7E151628AED2A6ABF715880$
1	$RC_1 = 0x9CF4F3C762E7160F38B4DA56$
2	$RC_2 = 0xA784D9045190CFEF324E7738$
3	$RC_3 = 0x926CFBE5F4BF8D8D8C31D763$
4	$RC_4 = 0xDA06C80ABB1185EB4F7C7B57$
5	$RC_5 = 0x57F5958490CFD47D7C19BB42$
6	$RC_6 = 0x158D9554F7B46BCED55C4D79$

7	$RC_7 = 0xFD5F24D6613C31C3839A2DDF$
8	$RC_8 = 0x8A9A276BCFBFA1C877C56284$
9	$RC_9 = 0xDAB79CD4C2B3293D20E9E5EA$
10	$RC_{10} = 0xF02AC60ACC93ED874422A52E$
11	$RC_{11} = 0xCB238FEEE5AB6ADD835FD1A0$
12	$RC_{12} = 0x753D0A8F78E537D2B95BB79D$
13	$RC_{13} = 0x8DCAEC642C1E9F23B829B5C2$
14	$RC_{14} = 0x780BF38737DF8BB300D01334$
15	$RC_{15} = 0xA0D0BD8645CBFA73A6160FFE$
16	$RC_{16} = 0x393C48CBBBCA060F0FF8EC6D$
17	$RC_{17} = 0x31BEB5CCEED7F2F0BB088017$
18	$RC_{18} = 0x163BC60DF45A0ECB1BCD289B$
19	$RC_{19} = 0x06CBBFEA21AD08E1847F3F73$
20	$RC_{20} = 0x78D56CED94640D6EF0D3D37B$
21	$RC_{21} = 0xE67008E186D1BF275B9B241D$
22	$RC_{22} = 0xEB64749A47DFDFB96632C3EB$
23	$RC_{23} = 0x061B6472BBF84C26144E49C2$
24	$RC_{24} = 0xD04C324EF10DE513D3F5114B$
25	$RC_{25} = 0x8B5D374D93CB8879C7D52FFD$
26	$RC_{26} = 0x72BA0AAE7277DA7BA1B4AF14$
27	$RC_{27} = 0x88D8E836AF14865E6C37AB68$
28	$RC_{28} = 0x76FE690B571121382AF341AF$
29	$RC_{29} = 0xE94F77BCF06C83B8FF5675F0$
30	$RC_{30} = 0x979074AD9A787BC5B9BD4B0C$
31	$RC_{31} = 0x5937D3EDE4C3A79396215EDA$

Table A-4. Round constants for $w = 128$, rounds 0~39

Round t	Round constant RC
0	$RC_0 = 0xB7E151628AED2A6ABF7158809CF4F3C7$

1	$RC_1 = 0x62E7160F38B4DA56A784D9045190CFEF$
2	$RC_2 = 0x324E7738926CFBE5F4BF8D8D8C31D763$
3	$RC_3 = 0xDA06C80ABB1185EB4F7C7B5757F59584$
4	$RC_4 = 0x90CFD47D7C19BB42158D9554F7B46BCE$
5	$RC_5 = 0xD55C4D79FD5F24D6613C31C3839A2DDF$
6	$RC_6 = 0x8A9A276BCFBFA1C877C56284DAB79CD4$
7	$RC_7 = 0xC2B3293D20E9E5EAF02AC60ACC93ED87$
8	$RC_8 = 0x4422A52ECB238FEEE5AB6ADD835FD1A0$
9	$RC_9 = 0x753D0A8F78E537D2B95BB79D8DCAEC64$
10	$RC_{10} = 0x2C1E9F23B829B5C2780BF38737DF8BB3$
11	$RC_{11} = 0x00D01334A0D0BD8645CBFA73A6160FFE$
12	$RC_{12} = 0x393C48CBBBCA060F0FF8EC6D31BEB5CC$
13	$RC_{13} = 0xEED7F2F0BB088017163BC60DF45A0ECB$
14	$RC_{14} = 0x1BCD289B06CBBFEA21AD08E1847F3F73$
15	$RC_{15} = 0x78D56CED94640D6EF0D3D37BE67008E1$
16	$RC_{16} = 0x86D1BF275B9B241DEB64749A47DFDFB9$
17	$RC_{17} = 0x6632C3EB061B6472BBF84C26144E49C2$
18	$RC_{18} = 0xD04C324EF10DE513D3F5114B8B5D374D$
19	$RC_{19} = 0x93CB8879C7D52FFD72BA0AAE7277DA7B$
20	$RC_{20} = 0xA1B4AF1488D8E836AF14865E6C37AB68$
21	$RC_{21} = 0x76FE690B571121382AF341AFE94F77BC$
22	$RC_{22} = 0xF06C83B8FF5675F0979074AD9A787BC5$
23	$RC_{23} = 0xB9BD4B0C5937D3EDE4C3A79396215EDA$
24	$RC_{24} = 0xB1F57D0B5A7DB461DD8F3C75540D0012$
25	$RC_{25} = 0x1FD56E95F8C731E9C4D7221BBED0C62B$
26	$RC_{26} = 0xB5A87804B679A0CAA41D802A4604C311$
27	$RC_{27} = 0xB71DE3E5C6B400E024A6668CCF2E2DE8$
28	$RC_{28} = 0x6876E4F5C50000F0A93B3AA7E6342B30$
29	$RC_{29} = 0x2A0A47373B25F73E3B26D569FE2291AD$

30	$RC_{30} = 0x36D6A147D1060B871A2801F978376408$
31	$RC_{31} = 0x2FF592D9140DB1E9399DF4B0E14CA8E8$
32	$RC_{32} = 0x8EE9110B2BD4FA98EED150CA6DD89322$
33	$RC_{33} = 0x45EF7592C703F532CE3A30CD31C070EB$
34	$RC_{34} = 0x36B4195FF33FB1C66C7D70F93918107C$
35	$RC_{35} = 0xE2051FED33F6D1DE9491C7DEA6A5A442$
36	$RC_{36} = 0xE154C8BB6D8D0362803BC248D414478C$
37	$RC_{37} = 0x2AFB07FFE78E89B9FECA7E3060C08F0D$
38	$RC_{38} = 0x61F8E36801DF66D1D8F9392E52CAEF06$
39	$RC_{39} = 0x53199479DF2BE64BBAAB008CA8A06FDA$